RGB BLE Controller

Android Application — Technical Report

Wireless RGB LED Control over Bluetooth Low Energy with ESP32

0. Glossary of Acronyms

Before diving in, here is a plain-language definition of every abbreviation used throughout this report.

Term	Full Name	What it means in plain language
BLE	**Bluetooth Low Energy**	A version of Bluetooth designed to consume very little battery. Used for short-range wireless links (up to ~10 m) where only small amounts of data are sent periodically.
GATT	**Generic Attribute Profile**	The set of rules that define how two BLE devices exchange data. One device is the server (the ESP32, which holds the data), the other is the client (the phone, which reads/writes it).
UUID	**Universally Unique Identifier**	A 128-bit number used to identify a specific service or characteristic on a BLE device — like a unique address label for each data channel.
NUS	**Nordic UART Service**	A popular open BLE profile created by Nordic Semiconductor that mimics a simple serial (UART) cable over BLE. Provides one channel to send data and one to receive.
UART	**Universal Asynchronous Receiver-Transmitter**	A very common hardware serial communication standard. Data is sent one bit at a time using just two wires (TX and RX). NUS emulates this over BLE.
API	**Application Programming Interface**	A set of functions and rules provided by the operating system or a library that programmers can call to use a feature — here it refers to Android's BLE functions.
ATT	**Attribute Protocol**	The low-level BLE protocol that defines how data is stored and exchanged between devices. GATT is built on top of ATT.
MTU	**Maximum Transmission Unit**	The largest chunk of data that can be sent in a single BLE packet. Default is 23 bytes; 3 bytes are overhead, leaving 20 bytes for payload.
RGB	**Red, Green, Blue**	The three primary colour channels used to mix any visible colour. Each channel is a value from 0 (off) to 255 (full intensity).

SDK	**Software Development Kit**	A collection of tools, libraries, and documentation that makes it easier to build apps for a specific platform (here, Android).
HSV	**Hue, Saturation, Value**	An alternative way to describe colour that is more intuitive for humans. Hue is the colour angle (0-360°), Saturation is how vivid, Value is brightness.
UI	**User Interface**	The part of the app the user sees and interacts with — buttons, sliders, colour wheel, and preview.
GATT Client/Server	**GATT Client / GATT Server**	In BLE, the server holds the data (ESP32). The client connects and requests to read or write that data (Android phone). This report always uses the phone as client.

1. System Overview

The RGB BLE Controller is an Android application written in Java that enables wireless control of RGB LEDs connected to an ESP32 microcontroller over Bluetooth Low Energy (BLE). The application provides a real-time colour-picking interface with brightness adjustment and streams 8-bit RGB values to the ESP32 via a UART-over-BLE Nordic UART Service (NUS).

1.1 Architecture

The Android application was developed in Java targeting API (Application Programming Interface) levels 21 through 36, with explicit handling of the Android 12 (API 31) BLE permission model, which means that the application is capable of running on any Android phone with version 5.0 (API 21) to 16 (API 31 – current Android Version).

The ESP32 firmware was written in Arduino C++ using the ESP32 BLE Arduino library.

The system has three physical layers:

Layer	Hardware / Software	Responsibility
Presentation	Android App (Java)	Colour wheel UI, brightness seekbar, live preview, BLE connection management
Communication	BLE (GATT over NUS profile)	Wireless transport of RGB values from phone to ESP32
Actuation	ESP32 + RGB LED	Receives serial RGB command and drives the LED accordingly

The Android app acts as a GATT Client: it scans for the ESP32, connects, discovers its services, and writes colour data to the write characteristic. The ESP32 acts as a GATT Server: it advertises its NUS service, accepts the connection, and reads incoming data.

1.2 Communication Protocol

The BLE link uses the Nordic UART Service (NUS), a widely adopted open profile that emulates a bidirectional serial cable. The app only uses the TX direction (phone → ESP32).

Parameter	Value
Service UUID	6E400001-B5A3-F393-E0A9-E50E24DCCA9E — this is the fixed NUS service identifier
Write Characteristic UUID	6E400002-B5A3-F393-E0A9-E50E24DCCA9E — the phone writes RGB data here
Write type	WRITE_TYPE_DEFAULT — the ESP32 sends an acknowledgement back for each packet
Payload encoding	ASCII text, format: "R,G,B\n" — e.g. "255,0,128\n"
Max payload per packet	20 bytes (ATT MTU 23 bytes minus 3 bytes ATT overhead)
Actual max payload used	12 bytes ("255,255,255\n") — well within the 20-byte limit

> **Why ASCII?** Sending "255,0,128\n" instead of raw binary bytes makes the ESP32 firmware much simpler: it reads a line of text and splits it on commas. No custom binary parser needed.

1.3 Data Flow Diagram

The path from finger movement to LED colour change:

```
User drags colour wheel / moves brightness seekbar
        |
        v
sendRGB() — posts 50 ms debounce runnable
        |
        v  (after 50 ms with no new input)
sendRGBNow() — applies brightness, formats ASCII string
        |
        v
BluetoothGatt.writeCharacteristic()  [phone → BLE radio]
        |
        v  (wireless, ~10 ms latency)
ESP32 NUS RX characteristic receives "R,G,B\n"
        |
        v
ESP32 parses R, G, B values and drives RGB LED
```

End-to-end data flow from user input to LED output

2. Application Structure

2.1 MainActivity

MainActivity is the single-activity entry point and implements all BLE logic, UI event handling, and colour management. It implements the ColorWheelView.OnColorChangeListener interface to receive colour updates from the custom wheel widget.

2.2 State Model

Three volatile boolean flags guard concurrent BLE operations:

- isConnected — true while a GATT connection is active
- isScanning — true during active BLE scan
- isWriting — true between writeCharacteristic() call and onCharacteristicWrite() callback, preventing concurrent writes that would corrupt the GATT queue

2.3 Threading Model

The application uses two Handler instances anchored to the main looper:

- mainHandler — schedules the scan timeout runnable (10 s) and routes GATT callbacks back to the UI thread via runOnUiThread()
- sendHandler — implements a 50 ms debounce on outgoing RGB writes to coalesce rapid colour-wheel or seekbar events into single GATT writes

3. Implementation

This section walks through the main building blocks of the Android application code, with excerpts from the source to illustrate each point.

3.1 BLE Scanning & Device Discovery

The app scans for BLE devices and stops as soon as it finds one named "ESP32_RGB**"(nome exemplo_podemos ainda mudar de acordo com o nome do Bluetooth do esp32)**. A 10-second timeout prevents the scan from running indefinitely if the device is out of range.

```
// ScanCallback — called once per nearby BLE advertisement
public void onScanResult(int callbackType, ScanResult result) {
    BluetoothDevice device = result.getDevice();
    String name = device.getName();
    if (name != null && name.equals(DEVICE_NAME)) {  // "ESP32_RGB"
        stopBleScan();       // stop immediately — saves battery
        connectToDevice(device);
    }
}
```

Listing 1 — ScanCallback matches device by advertised name

Lazy scanner init: bleScanner is obtained from the BluetoothAdapter only after the user grants BLE permissions. If it were resolved at app start, it would return null on first launch before permissions are accepted, crashing the app.

```
// ensureScanner() — resolves bleScanner lazily
private boolean ensureScanner() {
    if (bleScanner != null) return true;
    if (bluetoothAdapter == null) return false;
    bleScanner = bluetoothAdapter.getBluetoothLeScanner();
    return bleScanner != null;
}
```

Listing 2 — Lazy initialisation prevents null crash on first launch

3.2 GATT Connection & Service Discovery

Once the device is found, the app opens a GATT connection. GATT is the protocol layer that organises data into Services and Characteristics. The app must first discover which services the ESP32 offers before it can write any data.

```
// Step 1: open GATT connection
bluetoothGatt = device.connectGatt(this, false, gattCallback);
//                                        ^--- autoConnect=false
//  false = connect immediately (faster); true = background reconnect

// Step 2: discover services once connected
public void onConnectionStateChange(BluetoothGatt gatt, int status, int newState) {
    if (newState == BluetoothProfile.STATE_CONNECTED) {
        gatt.discoverServices();  // triggers onServicesDiscovered()
    }
}

// Step 3: find the write characteristic by UUID
public void onServicesDiscovered(BluetoothGatt gatt, int status) {
    BluetoothGattService service = gatt.getService(SERVICE_UUID);
    writeCharacteristic = service.getCharacteristic(CHAR_WRITE_UUID);
}
```

Listing 3 — Three-step GATT connection sequence

autoConnect() is set to false because true causes exponential back-off delays on many Android versions — the initial connection attempt can take 30+ seconds. With false, the OS connects within 1–3 seconds.

3.3 Data Transmission — Debounce & Write Guard

Every colour or brightness change triggers sendRGB(). Because the user can move the slider dozens of times per second, sending a BLE packet on every event would overflow the GATT write queue. Two mechanisms prevent this:

— only the last event within any 50 ms window is actually sent.

— the GATT stack allows only one outstanding write at a time; a second call before the first completes silently fails. The guard blocks new writes until the ESP32 acknowledges the previous one.

```
// Debounce: cancel any pending send, post a fresh one 50 ms later
private void sendRGB() {
    sendHandler.removeCallbacks(sendRunnable);
    sendHandler.postDelayed(sendRunnable, SEND_DEBOUNCE_MS); // 50 ms
}

// Actual write — only executes if not already writing
private void sendRGBNow() {
    if (isWriting) return;  // write already in progress, skip

    int r = (Color.red(currentColor)   * brightness) / 255;
    int g = (Color.green(currentColor) * brightness) / 255;
    int b = (Color.blue(currentColor)  * brightness) / 255;

    String data = r + "," + g + "," + b + "\n";  // e.g. "255,0,128\n"
    byte[] dataBytes = data.getBytes();

    // API 33+ uses new overload; older uses deprecated setValue()
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        bluetoothGatt.writeCharacteristic(writeCharacteristic,
            dataBytes, BluetoothGattCharacteristic.WRITE_TYPE_DEFAULT);
    } else {
        writeCharacteristic.setValue(dataBytes);
        bluetoothGatt.writeCharacteristic(writeCharacteristic);
    }
    isWriting = true;  // cleared in onCharacteristicWrite()
}
```

Listing 4 — Debounced, guarded BLE write

Brightness formula: R_out = (R_wheel × brightness) / 255. This integer-only formula avoids floating-point math in the hot path. At brightness=128 a full red (255,0,0) becomes (128,0,0).

3.4 Disconnection & Resource Cleanup

BLE resources on Android must be released in a specific order to avoid memory leaks or crash-inducing double-close errors. The app uses a two-phase pattern:

- Phase 1 — Call gatt.disconnect(). This triggers the OS to send a BLE disconnection request to the ESP32 and waits for confirmation.
- Phase 2 — Only after the GATT callback confirms STATE_DISCONNECTED is gatt.close() called to free the OS resource.

```
// Phase 1: request disconnection
private void disconnectBle() {
    bluetoothGatt.disconnect();
}

// Phase 2: close only after the callback fires
public void onConnectionStateChange(BluetoothGatt gatt, int status, int newState) {
    if (newState == BluetoothProfile.STATE_DISCONNECTED) {
        isConnected = false;
        writeCharacteristic = null;
        closeGattInternal();  // calls gatt.close() + nulls reference
    }
}

private void closeGattInternal() {
    if (bluetoothGatt != null) {
        bluetoothGatt.close();
        bluetoothGatt = null;
    }
}
```

Listing 5 — Two-phase GATT cleanup prevents double-close race

3.5 Android API Compatibility

The app supports both modern (API 33+, Android 13+) and older Android devices. The two main divergence points are BLE permissions and the writeCharacteristic() call:

Feature	API < 31 (Android 11 and below)	API 31+ (Android 12+)
BLE permissions	Declared in manifest only (BLUETOOTH, BLUETOOTH_ADMIN)	Runtime request required: BLUETOOTH_SCAN + BLUETOOTH_CONNECT
writeCharacteristic()	Deprecated: setValue() then writeCharacteristic(char)	New overload: writeCharacteristic(char, bytes, writeType) returns status int

4. Strengths & Limitations

4.1 Strengths

Robust concurrency handling

The three volatile flags (isConnected, isScanning, isWriting) combined with the two Handler instances provide clear, safe multi-thread coordination between the BLE callback thread and the UI thread — a common source of crashes in Android BLE apps.

```
private volatile boolean isConnected = false;
private volatile boolean isScanning  = false;
private volatile boolean isWriting   = false;
// volatile: JVM guarantees all threads see the latest value immediately
```

Listing 6 — Volatile flags ensure thread-safe state visibility

Clean lifecycle management

onDestroy() cancels all pending runnables and disconnects GATT only if actually connected, preventing resource leaks when the user navigates away from the app.

Backward compatibility

A single code path supports Android API levels from legacy (below 31) through modern (API 33+) by branching on Build.VERSION.SDK_INT. This maximises the addressable device base without maintaining separate builds.

Event-driven, non-blocking design

No busy-wait loops or polling. Every state transition (connected, services discovered, write complete) is handled by a callback, keeping the UI thread free and the app responsive at all times.

Energy efficiency

The BLE scan stops the moment the target device is found. The 50 ms debounce means at most 20 BLE packets per second even during rapid slider movement — well within what the BLE radio can sustain without draining the phone battery.

3.2 Limitations

No write-acknowledgement timeout

When isWriting is set to true, it is only cleared by the onCharacteristicWrite() callback. If the BLE link drops mid-write (e.g., the ESP32 resets), the callback never fires and the app silently stops sending any further colour updates — stuck in a "writing" state.

```
// Current code — no watchdog
if (isWriting) return;  // if callback never comes, this blocks forever

// Improvement: post a 2-second timeout to clear isWriting
// mainHandler.postDelayed(() -> { isWriting = false; }, 2000);
```

Listing 7 — Missing write watchdog (suggested fix shown as comment)

Single-device scan

The scanner picks the first device named exactly "ESP32_RGB" and connects immediately. There is no way to choose between multiple ESPs in range, nor to reconnect automatically after an unexpected drop.

No MTU negotiation

The app uses the default ATT MTU (23 bytes, 20 usable). Calling gatt.requestMtu(185) after connection would allow larger payloads and reduce overhead if the protocol were extended to send additional data fields.

ASCII payload overhead

"255,255,255\n" is 12 bytes. The same information in binary would be 3 bytes. For RGB-only control this is irrelevant, but if the protocol were expanded (e.g., adding animation speed or brightness as separate fields), the ASCII format would start eating into the 20-byte ATT limit.

5. Biggest Challenges

5.1 Android BLE Permission Model Changes

Android 12 (API 31) introduced BLUETOOTH_SCAN and BLUETOOTH_CONNECT as runtime permissions — meaning the user must approve a pop-up during the first use. Before API 31, BLE permissions only needed to be declared in the app manifest. Writing code that gracefully handles both worlds — requesting runtime permissions on newer devices while not crashing on older ones — requires careful version checks at every entry point.

```
private boolean needsBluetoothPermissions() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) { // API 31
        return ActivityCompat.checkSelfPermission(this,
            Manifest.permission.BLUETOOTH_SCAN) != PERMISSION_GRANTED
            || ActivityCompat.checkSelfPermission(this,
            Manifest.permission.BLUETOOTH_CONNECT) != PERMISSION_GRANTED;
    }
    return false; // legacy devices need no runtime check
}
```

Listing 8 — Version-aware permission check

5.2 GATT Write Queue Overflow

The BLE GATT stack on Android processes writes one at a time. If writeCharacteristic() is called while a previous write is still pending, the call silently returns false and the data is lost. This is not obvious from the API documentation, and discovering it required observing dropped colour updates when the colour wheel was dragged quickly. The solution was the isWriting guard combined with the debounce, which together ensure at most one outstanding write at any moment.

5.3 Race Condition on GATT Close

Calling gatt.close() immediately on disconnection can race against the OS completing the teardown, sometimes leaving the underlying Bluetooth socket in a half-open state. The correct pattern — wait for STATE_DISCONNECTED in the callback before calling close() — is not prominently documented and was found only after observing intermittent connection failures on the second attempt to connect after a disconnect.

5.4 Null bleScanner on First Launch

getBluetoothLeScanner() returns null if called before BLE permissions are granted, or before Bluetooth is enabled. On first launch, the app requests permissions; if the user grants them and then immediately presses Connect, bluetoothAdapter.getBluetoothLeScanner() might still return null because the system has not yet propagated the permission grant. The lazy ensureScanner() method resolves this by deferring the scanner acquisition to the moment it is actually needed.

5.5 Deprecated API Handling

The writeCharacteristic(BluetoothGattCharacteristic) overload was deprecated in API 33. The replacement requires a different method signature and returns an integer status code instead of a boolean. Maintaining a single codebase that compiles and runs correctly on both old and new Android versions while correctly using the right API for each requires an explicit SDK version branch — adding code complexity that exists purely to satisfy the Android compatibility layer.

6. Conclusion

The RGB BLE Controller demonstrates a complete end-to-end embedded wireless system: a custom Android UI communicates colour information to an ESP32 microcontroller using the Bluetooth Low Energy protocol, all within a clean, event-driven architecture.

The choice of BLE over classic Bluetooth is well justified — the data volume is tiny (12 bytes per update), and BLE imposes negligible battery cost while offering adequate range. The NUS (Nordic UART Service) profile is an ideal fit because it requires no custom GATT profile design and is natively supported by most ESP32 BLE libraries, keeping the firmware side minimal.

On the Android side, the implementation handles the main pitfalls of Android BLE development: the API 31+ runtime permission split, the GATT write queue overflow problem, the two-phase close race, and the deprecation of the old writeCharacteristic API. The use of volatile flags, a debounce handler, and callback-driven state transitions results in a robust, non-blocking design that remains responsive even under rapid user input.

The main areas left for improvement are a write-acknowledgement watchdog (to recover from mid-write link loss), automatic reconnection after an unexpected disconnect, and MTU negotiation if the protocol is extended. None of these are blocking issues for the current use case — controlling a single RGB LED from close range — but they would be essential before deploying this in a production product.